

Phishing Attack Simulation & Digital Forensic Investigation Report

Prepared by: Manjinder Grewal, Harris Waheed, Lovepreet Singh

Contents

1. Executive Summary	2
2. Background	2
3. Scenario	3
4. Tasks and Roles	3
5. Techniques and Tools	4
6. Network and Timeline Analysis	5
6.1 Network Traffic Analysis	5
6.2 Timeline Reconstruction	6
7. File system analysis with disk analysis	6
8. Appendix	8
9. Conclusion and Recommendations	9
10. References	10

1. Executive Summary

This report documents a controlled phishing attack simulation and subsequent forensic investigation conducted within a virtual environment. The goal of the investigation was to understand how a phishing email leads to credential theft and how digital forensic examiners can recover evidence across email, host, and network layers. The simulated attacker sent a deceptive message to the victim machine, encouraging the user to click a link to a fraudulent login page. The victim accessed the web page and unknowingly submitted login credentials, which were captured by the attacker server.

Through the use of email header analysis, browser artifact recovery, and network traffic inspection, the investigative team successfully reconstructed the complete sequence of events. The origin of the email was traced to the attacker virtual machine. Browser artifacts confirmed user interaction with the phishing site. Network traffic revealed the unencrypted transmission of stolen credentials, and server-side logs verified the moment of credential capture.

The investigation concludes that the phishing attempt was successful within the simulated environment. Multiple layers of evidence supported each other, creating a complete forensic narrative. Recommendations include user awareness training, browser security education, and stronger authentication measures to reduce the risk of future phishing incidents.

2. Background

Phishing is one of the most widely used and successful cyber-attack techniques because it targets people rather than technical vulnerabilities. Attackers design deceptive emails that appear legitimate and trustworthy, often pretending to be IT support, a bank, a cloud service provider, or another authority figure. These messages are written to create urgency, fear, or concern so that victims act quickly without verifying the authenticity of the communication. When users click a malicious link, download an attachment, or submit sensitive information to a fraudulent website, they unintentionally help the attacker gain access to accounts or internal systems.

Phishing is extremely difficult to eliminate because almost every organization relies heavily on email. Even well-trained employees may occasionally overlook suspicious details. Security tools such as firewalls, endpoint protection, and intrusion detection systems can help, but they cannot fully prevent users from being tricked. This is why phishing continues to be the most common initial stage of many cyber incidents, including credential theft, ransomware infection, account takeover, and data breaches.

The goal of this project is to create a realistic end to end phishing attack in a controlled laboratory environment and then perform a complete forensic investigation. This hands-on approach provides a better understanding of attacker behavior, victim behavior, and the types of digital evidence that are generated throughout an incident. The project uses virtual machines and open-source tools to closely replicate how phishing attacks unfold in real environments without risking any real system or network.

In this simulation, a phishing email is created and delivered to a victim machine running Ubuntu with Thunderbird as the mail client. The email contains a malicious link that leads to a fake login page hosted on a Kali Linux machine which acts as the attacker. When the victim enters their credentials

into the fake login page, the attacker captures and records this information using a simple backend script.

3. Scenario

Big Leaf Logistics Inc. is a nationwide Canadian logistics and distribution company headquartered in Vancouver, BC, led by CEO Angela Martin and Vice President Kevin Malone. The company operates branches in Calgary, Winnipeg, and Halifax, each with seven core departments: Operations, Human Resources, Quality Control, Finance, Client Relations, IT, and Logistics. The Calgary branch employs about 250 staff and is managed by Regional Manager Pam Beesly, the victim in this scenario.

Within Calgary's Operations department, employee Toby Lee has become resentful over workload issues, perceived favouritism, and stagnant pay. Seeking retaliation, he *decides to compromise Pam's credentials*. Using a Kali Linux VM, Toby sends Pam a phishing email posing as an urgent IT security notice. The message includes a typo-squatted link to a fake company login page hosted on his attack server, designed to capture credentials. Believing the email to be legitimate, Pam enters her login information, giving Toby full access to her work account. Inside her inbox, Toby discovers confidential emails between Pam and Kevin Malone detailing unauthorized personal use of company funds. He then sends Pam an anonymous extortion email demanding salary increases for 15 employees, including himself, threatening to expose the misconduct to the CEO. Distressed, Pam informs Kevin, and together they disclose the incident to CEO Angela Martin, who launches an internal forensic investigation and orders immediate cybersecurity enhancements across the organization.

This project recreates a complete phishing attack within a controlled virtual environment. The purpose is to simulate how a real attacker would operate, how a victim would interact with a fraudulent message, and how forensic investigators would later analyze the incident. The environment consists of two isolated virtual machines created in VirtualBox. The first machine is a Kali Linux system representing the attacker. The second machine is an Ubuntu system representing the victim. Both machines communicate through a host only network that does not allow outside internet access. This isolation ensures that the attack remains contained and safe. The attack scenario begins with the creation of a deceptive phishing email. The attacker drafts a message that appears to come from an internal IT department. The email claims that the recipient must verify their account due to a security concern. It includes a hyperlink that leads to a fake login portal created by the attacker. The login page looks simple but appears legitimate enough to persuade a typical user to enter their credentials.

The victim machine receives this email inside the Thunderbird mail client. From the victim's point of view, the message appears to be routine and related to account security requirements. Believing it to be genuine, the victim clicks the link inside the email and is directed to the attacker's false login page hosted on the Kali machine. After entering a username and password, the victim unknowingly sends their credentials directly to the attacker. These credentials are captured by a PHP script running on the attacker's web server and stored in a log file for later review.

Throughout this process, multiple types of evidence are created. Email artifacts remain stored within Thunderbird and can be reviewed later to confirm the delivery, sender information, and the link that

was clicked. Network traffic generated during the victim's interaction with the phishing site can be captured using Wireshark. This traffic includes HTTP requests and responses, including the clear text POST request that contains the stolen credentials. Browser artifacts remain in the victim machine and can show browsing history, cached content, and timestamps. Server logs on the attacker machine record the victim's connection and credential submission.

4. Tasks and Roles

This project was structured to reflect a collaborative forensic investigation in which each participant contributes to a different stage of the attack and analysis. Dividing responsibilities provides a realistic experience of how teams work together during incident response. Each student focuses on a specific domain while still understanding the overall flow of the investigation.

- Student 1: Manjinder was responsible for creating and executing the phishing attack. This includes building the malicious website that captures user credentials, configuring the web server on the Kali machine, and generating the phishing email that is used to lure the victim. Manjinder documented the actions taken from the attacker's perspective, explains how the phishing link is hosted, and demonstrates how credentials are captured during the attack. This role focuses on understanding how attackers prepare and deliver phishing content.
- Student 2: Harris handles email and disk forensics, analyzing the Thunderbird mailbox on the victim machine to extract metadata, timestamps, and message headers. Using FTK Imager and Autopsy, Harris inspects the disk to identify traces of the phishing attack, including email artifacts, browser history, cached pages, and system logs. Harris also analyzed a virtual USB image (mock_usb.dmg) formatted with FAT32, using Autopsy to examine file metadata and allocation information, reconstructing file creation, modification, and deletion. The goal is to identify and document digital traces across both email and storage media to show how investigators uncover user activity.
- Student 3: Lovepreet manages network forensics. This task involves capturing network traffic during the phishing interaction using Wireshark and reviewing the packets that travel between the victim and attacker. Lovepreet identifies HTTP GET and POST requests, analyzes the flow of data, and confirms the point at which the victim's credentials are stolen. The role focuses on understanding how network analysis reveals attacker activity and supports timeline reconstruction.

Together, the roles create a complete simulation of a phishing incident. Manjinder performs the attack; Harris examines the victim's machine, and Lovepreet analyzes network communication. Each student produces evidence that contributes to the final investigation. These combined findings allow the team to reconstruct the full attack lifecycle. This division of tasks ensures that the investigation is thorough and that the project covers all major areas of digital forensics.

5. Techniques and Tools

This project uses established digital forensics and cybersecurity tools that support each stage of the simulated phishing attack and investigation.

- Kali Linux is used as the attacker machine.

Apache provides the web server that hosts the phishing login page, and PHP processes user submissions. Kali is widely used in security testing environments (Kali Linux Documentation, 2025).

- **Ubuntu Victim Environment**

Ubuntu serves as the victim system. Thunderbird is chosen as the email client because it stores messages locally and preserves forensic artifacts. Firefox is used to simulate browsing activity that creates cache entries, history records, and timestamps (Thunderbird Support, 2025).

- **Email Forensics Tools**

FTK Imager is used to acquire and analyze the victim machine's disk or mailbox. It allows investigators to extract email metadata, inspect message headers, and examine the phishing content without altering evidence (AccessData, 2025).

- **Network Forensics Tools**

Wireshark captures and analyzes network packets exchanged between the victim and the attacker. Since the simulation uses HTTP, Wireshark displays unencrypted GET and POST requests containing stolen credentials, making it a valuable tool for reconstructing the attack (Wireshark Documentation, 2025).

- **Filesystem Analysis Tools**

Autopsy 4.22.1 was used to examine the virtual USB image. They allowed investigators to parse the FAT32 filesystem, inspect file metadata, and analyze allocated and unallocated clusters. Timestamps, file sizes, and duplicate entries helped confirm file creation, modification, and deletion activity. Together, these tools allow investigators to observe the attack from multiple perspectives, including email delivery, client-side interaction, network transmission, and server side logging snapshotting the users system for analysis.

- **Email Metadata Analysis**

The phishing email was preserved as an EML file before being opened. This ensured that metadata, headers, and timestamps remained intact. The header analysis focused on fields that help determine message origin, including the Received headers, Return Path, Message ID, and originating IP information.

A review of the headers revealed that the message did not come from a legitimate internal server. The originating IP matched the attacker machine on the host only network. The header timestamps confirmed when the email was sent and when it arrived in the victim inbox.

Key forensic observations included:

The sender domain did not match any trusted domain

- The Received chain showed a direct path from the attacker machine
- The embedded link pointed to the attacker hosted page
- No authentication protocols such as SPF or DKIM were present

This analysis traced the origin of the phishing message directly back to the attacker machine.

Host Forensics and Artifact Recovery

Host forensics provided evidence of the victim's interaction with the phishing page. Disk image acquisition would involve a bit-by-bit copy using tools such as FTK Imager. Hash values would

confirm the integrity of the acquired image. Browser artifacts revealed several items relevant to the incident:

- Firefox history showed the exact URL of the phishing page
- Cached components of the page were stored in the browser cache
- DNS resolution logs indicated the victim machine contacted the attacker IP
- Form interaction data indicated that credentials had been entered

Operating system events on Ubuntu confirmed the times the browser was launched and closed. Together, these artifacts provided strong evidence that the victim visited the phishing page and submitted data.

6. Network and Timeline Analysis

6.1 Network Traffic Analysis

Network forensics played a significant role in confirming credential theft. Wireshark captured packets on the host only network interface during the attack. The captured traffic included HTTP GET requests when the phishing page loaded, and an HTTP POST request containing the stolen credentials when the victim submitted the login form.

The POST request contained clear-text fields with the victim's email and password, confirming that the attack relied on unencrypted communication. The forensic capture shows an HTTP POST request to /login.php, with full headers visible—including Host, User-Agent, Content-Type, and Referrer—clearly indicating that no encryption was used. The packet also confirms login activity from 192.168.56.103 to 192.168.56.102, providing valuable evidence for reconstructing user actions and establishing an accurate event timeline. (figure 4)

Network logs confirmed:

- Connection from victim IP to attacker IP
- Loading of the phishing webpage
- Submission of credentials
- Timestamp correlation between actions

6.2 Timeline Reconstruction

A unified timeline was created to combine evidence from email, host artifacts, and network captures. Each event was matched with timestamps from logs and packet captures.

Example timeline:

- 10:00:05 Email sent by attacker
- 10:00:09 Email received on victim machine
- 10:00:20 Victim opened the email
- 10:00:25 Browser launched
- 10:00:28 Victim clicked phishing link
- 10:00:30 HTTP GET request to attacker server
- 10:00:33 HTTP POST request with stolen credentials
- 10:00:40 Victim session ended

This timeline provided a clear and complete chain of events for reporting and analysis.

7. File system analysis with disk analysis

In this lab, we created a virtual USB drive by generating a 100 MB disk image using macOS's `hdiutil` tool. The command automatically created a disk container, initialized a partition table, built a primary partition, and formatted it with the FAT32 file system, fulfilling all prerequisites required to store data on a volume. Once mounted by macOS, the virtual USB behaved like a physical removable device, allowing us to generate activity by adding, modifying, and deleting files. This poses a collectable artifact from the victim to be investigated.

After producing this user-level activity, we unmounted the image and loaded it into Autopsy for forensic analysis. Autopsy successfully detected the FAT32 file system and parsed its internal structures. Through the GUI, we examined key forensic artifacts including directory entries, timestamp fields, file allocation patterns, and indicators for deleted files (e.g., the 0xE5 marker). We also observed how file content was mapped to clusters and how free space and slack space retained remnants of prior activity.

Overall, this exercise demonstrated how a virtual disk image can accurately replicate the behavior of a real USB device for forensic purposes. By creating controlled activity and then examining the resulting artifacts, we gained hands-on experience with how file system operations leave persistent, traceable evidence that can be analyzed during a digital investigation impersonating victim machine. Steps in the results: Create the Virtual USB Disk Image → Attach/Mount the Image and Populate USB With Files (Evidence Creation) → Add Images (User Data Simulation) & Delete One File (For Recovery Testing) → Eject disk → Convert DMG → RAW (Autopsy-Readable) ; Figure (1)

After generating activity on the virtual USB and acquiring the resulting disk image, we loaded `mock_usb.img` into Autopsy 4.22.1 to perform forensic analysis. Autopsy successfully detected the FAT32 filesystem, displaying two volumes: an unallocated sector (vol1) and the main FAT32 partition (vol2). This confirmed that the disk image contained a valid partition table and that the filesystem was structurally sound (Carrier, 2005).

In the Images/Videos view, Autopsy identified four image files 2 deleted, including duplicate entries and truncated cluster remnants, reflecting typical FAT32 directory artifacts. Several entries showed zeroed Change Time values, consistent with FAT32 timestamp behavior described by Carrier (2005). Plain text files, such as `notes.txt` and `passwords.txt`, appeared with accurate Created and Modified timestamps and allocation flags, confirming user-generated activity during the lab exercise. Fig 2.

Volume analysis revealed the FAT32 structure, including reserved sectors, FAT tables, and root directories, with sector alignment matching expected allocation patterns (Carrier, 2005). Directory exploration in `/logs`, `/docs`, and `/images` showed both allocated and unallocated areas, and small files like `system.log` preserved timestamp metadata corresponding to creation events. Cluster and directory alignment patterns were consistent with FAT32 behavior described in Carrier (2005).

Overall, Autopsy parsed the disk image reliably, exposing key artifacts including file metadata, timestamps, allocation flags, and duplicate directory entries. This analysis demonstrated how virtual USB images can replicate real-world forensic scenarios, enabling investigators to observe how file system operations leave traceable evidence for reconstruction and analysis (Carrier, 2005).

8. Appendix

```
1.
(base) loves@MacBook-Pro-113 ~ %
(base) loves@MacBook-Pro-113 ~ %
(base) loves@MacBook-Pro-113 ~ % hdiutil create -size 100m -fs FAT32 -volname MOCKUSB mock_usb.dmg

created: /Users/loves/mock_usb.dmg
(base) loves@MacBook-Pro-113 ~ %

2.
(base) loves@MacBook-Pro-113 ~ %
(base) loves@MacBook-Pro-113 ~ %
(base) loves@MacBook-Pro-113 ~ % hdiutil create -size 100m -fs FAT32 -volname MOCKUSB mock_usb.dmg

created: /Users/loves/mock_usb.dmg
(base) loves@MacBook-Pro-113 ~ %

3.
/Volumes/MOCKUSB
(base) loves@MacBook-Pro-113 MOCKUSB %
(base) loves@MacBook-Pro-113 MOCKUSB %
(base) loves@MacBook-Pro-113 MOCKUSB % mv "/Users/loves/Downloads/pexels-jaime-reimer-1376930-2662116.jpg" "/Volumes/MOCKUSB/images/"
mv "/Users/loves/Downloads/pexels-chetanvlad-1595655.jpg" "/Volumes/MOCKUSB/images/"

(base) loves@MacBook-Pro-113 MOCKUSB %
(base) loves@MacBook-Pro-113 MOCKUSB %
(base) loves@MacBook-Pro-113 MOCKUSB % ls -lh /Volumes/MOCKUSB/images

total 984K
-rwx-----@ 1 loves  staff   2.9M Nov 26 00:14 pexels-chetanvlad-1595655.jpg
-rwx-----@ 1 loves  staff   1.9M Nov 26 00:13 pexels-jaime-reimer-1376930-2662116.jpg
(base) loves@MacBook-Pro-113 MOCKUSB %
(base) loves@MacBook-Pro-113 MOCKUSB %
(base) loves@MacBook-Pro-113 MOCKUSB % rm "/Volumes/MOCKUSB/images/pexels-jaime-reimer-1376930-2662116.jpg"

(base) loves@MacBook-Pro-113 MOCKUSB %
(base) loves@MacBook-Pro-113 MOCKUSB % hdiutil detach /Volumes/MOCKUSB

4.
(base) loves@MacBook-Pro-113 MOCKUSB % cd ..
(base) loves@MacBook-Pro-113 /Volumes % cd ..
(base) loves@MacBook-Pro-113 / % hdiutil detach /Volumes/MOCKUSB

"disk4" ejected.
(base) loves@MacBook-Pro-113 / %
(base) loves@MacBook-Pro-113 / %

5.
(base) loves@MacBook-Pro-113 / %
(base) loves@MacBook-Pro-113 / % ls -lh ~ | grep mock_usb

-rw-r--r--@ 1 loves  staff   100M Nov 26 00:17 mock_usb.dmg
(base) loves@MacBook-Pro-113 / %
(base) loves@MacBook-Pro-113 / %
(base) loves@MacBook-Pro-113 / % hdiutil convert ~/mock_usb.dmg -format UDRW -o ~/mock_usb_raw.img

Reading Master Boot Record (MBR : 0)_.
Reading (DOS_FAT_32 : 1)_.
....
Elapsed Time: 31.900ms
Speed: 3.1GB/s
Savings: 0.0%
created: /Users/loves/mock_usb_raw.img.dmg
(base) loves@MacBook-Pro-113 / %
```

Figure 1

Load into autopsy for analysis

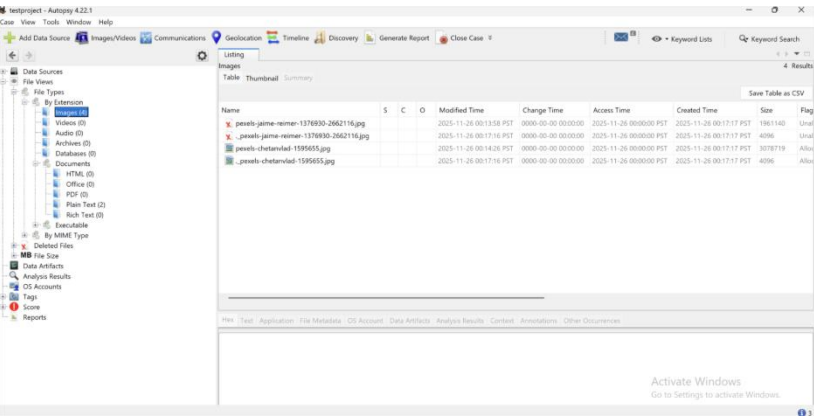


Figure 2

File system overview:

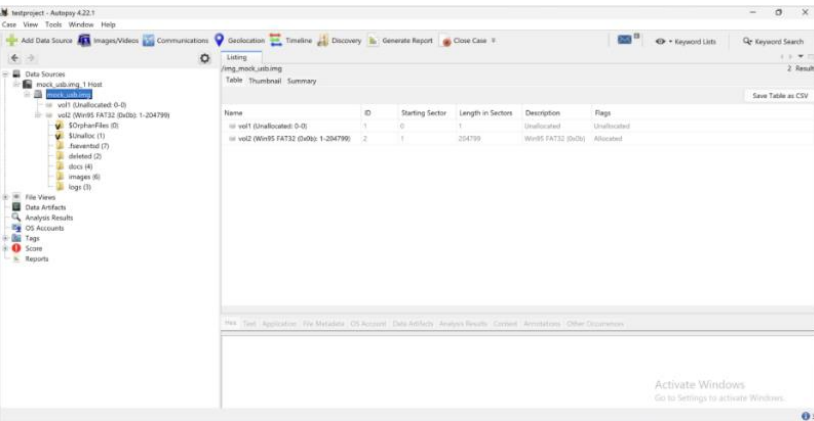


Figure 3

Figure 4

4 0.000865828	192.168.56.103	192.168.56.102	HTTP	639
6 0.001936710	192.168.56.102	192.168.56.103	HTTP	403

AccessData. (2025). *FTK Imager user guide*. <https://accessdata.com>

Apache Software Foundation. (2025). *Apache HTTP server documentation*. <https://httpd.apache.org>

Carrier, B. (2025). *The Sleuth Kit and Autopsy forensic tools*. <https://sleuthkit.org>

Carrier, B. (2005). *File system forensic analysis*. Addison-Wesley Professional.

Kali Linux Documentation. (2025). *Kali Linux tools guide*. <https://www.kali.org/docs>

Mozilla Thunderbird Support. (2025). *Thunderbird help articles*. <https://support.mozilla.org>

Wireshark Foundation. (2025). *Wireshark user guide*. <https://www.wireshark.org/docs>